

SECURITY AUDIT REPORT

Option D | Blockchain Technologies 2

1. Executive Summary

This report presents the results of an internal security audit performed by the our team. The protocol is an on-chain binary prediction market deployed on Arbitrum Sepolia. Users deposit USDC collateral to purchase YES or NO outcome tokens on questions like "Will ETH be above \$3000 by the end of 2025?". Token prices are determined by a constant-product automated market maker using the formula $x \text{ times } y \text{ equals } k$. When the resolution time arrives, a Chainlink price feed is consulted to determine the correct outcome. Users who hold winning tokens can redeem them for USDC at a one-to-one ratio. Trading fees of 0.3% are collected on every swap and sent to an ERC-4626 fee vault where liquidity providers can earn yield. The entire protocol is governed by PRED token holders through an OpenZeppelin Governor contract connected to a TimelockController with a mandatory two-day execution delay.

We used Slither static analysis tool running 101 detectors across 97 contracts, Foundry test suite with unit, fuzz, invariant and fork tests, and manual code review. The result is zero High severity findings and zero Medium severity findings in our own source files. All findings in the protocol code are Low or Informational severity.

Severity	Total Found	Fixed	Acknowledged
Critical	0	—	—
High	0	—	—
Medium	0	—	—
Low	3	2	1
Informational	5	0	5
Gas	2	1	1

2. Scope

2.1 Files In Scope

The audit covers the final submission commit on the main branch. All files reviewed are listed below.

- src/core/PredictionMarket.sol — the main market logic contract, UUPS upgradeable
- src/core/MarketAMM.sol — the constant-product AMM library with Yul assembly
- src/factories/MarketFactory.sol — factory contract using CREATE and CREATE2
- src/governance/PredictionGovernor.sol — OpenZeppelin Governor implementation
- src/governance/MarketTimelock.sol — TimelockController with two-day minimum delay
- src/oracles/OracleAdapter.sol — Chainlink price feed adapter with staleness check
- src/tokens/GovernanceToken.sol — ERC-20 token with voting power and permit support
- src/tokens/OutcomeToken.sol — ERC-1155 contract for YES and NO outcome tokens
- src/tokens/FeeVault.sol — ERC-4626 vault for LP fee accumulation

2.2 Files Out of Scope

The following files were not audited because they are either third-party libraries or test-only contracts that are never deployed to production networks.

- lib/openzeppelin-contracts and lib/openzeppelin-contracts-upgradeable — these are audited by OpenZeppelin
- lib/chainlink-brownie-contracts — Chainlink interfaces only
- test directory — Foundry test files, not deployed
- src/oracles/MockAggregator.sol — test mock only
- src/tokens/MockUSDC.sol — test mock only

3. Methodology

3.1 Tools Used

Tool	Purpose
Slither (101 detectors)	Automated static analysis for common vulnerability patterns
forge test	Unit, fuzz, invariant, and fork testing
forge coverage	Line coverage measurement across all source files
Manual review	Logic verification, state machine analysis, access control audit

3.2 Manual Review Approach

The manual review was divided into several phases. First we checked state machine correctness by verifying that all MarketState transitions from Open to PendingResolution to Resolved or Disputed are correct and cannot be bypassed. Every function that changes market state was checked against the enum definition to confirm no transition can be skipped or exploited.

Second we audited access control by checking every privileged function including createMarket, resolveMarket, pause, authorizeUpgrade, setMarket, and mint. We confirmed that each function checks the correct role and that roles are assigned correctly in the initialize function and constructors.

Third we verified the Checks-Effects-Interactions pattern in every external call. The most important case was buyOutcome which must update pool reserves before calling outcomeToken.mint, because ERC-1155 mint triggers the onERC1155Received callback on recipient contracts.

Fourth we reviewed integer arithmetic in the AMM formula. The calculation is amountIn times 997 times reserveOut divided by the sum of reserveIn times 1000 and amountIn times 997. With uint256 inputs and realistic reserve values, intermediate products stay within uint256 range. The Yul assembly version was compared against the pure Solidity reference using a fuzz test with 1000 runs.

Fifth we checked oracle integration including the staleness check that reverts if the price is older than 3600 seconds, the negative price check, and the answeredInRound comparison to prevent using data from stale aggregator rounds.

Sixth we documented the storage layout of both upgradeable contracts to prove that a V2 upgrade cannot cause storage collisions. OpenZeppelin upgradeable contracts use ERC-7201 namespaced storage which is provably disjoint from sequential Solidity slots.

4. Findings Table

ID	Title	Severity	Location	Status
L-01	createMarket not protected by nonReentrant	Low	PredictionMarket.sol line 108	Fixed
L-02	removeLiquidity allowed while market is Disputed	Low	PredictionMarket.sol line 163	Fixed
L-03	FeeVault.setMarket accepts zero address	Low	FeeVault.sol setMarket function	Acknowledged
I-01	Slither: block.timestamp usage in comparisons	Informational	Multiple files	Acknowledged

I-02	Slither: missing-inheritance IPredictionMarket	Informational	PredictionMarket.sol	Acknowledged
I-03	Slither: constable-states MockUSDC._decimals	Informational	MockUSDC.sol line 7	Acknowledged
I-04	Slither: immutable-states MockAggregator._decimals	Informational	MockAggregator.sol line 9	Fixed
I-05	Slither: unindexed event address parameters	Informational	OpenZeppelin library events	Acknowledged
G-01	MockUSDC._decimals should be constant	Gas	MockUSDC.sol line 7	Acknowledged
G-02	MockAggregator._decimals should be immutable	Gas	MockAggregator.sol line 9	Fixed

5. Detailed Findings

L-01 — createMarket Not Protected by nonReentrant

Severity: Low **Location:** src/core/PredictionMarket.sol, line 108 **Status:** Fixed

The createMarket function is protected by onlyRole and whenNotPaused modifiers but was missing the nonReentrant modifier. The function calls collateral.safeTransferFrom followed by outcomeToken.mint. If the collateral token used a non-standard implementation with transfer hooks similar to ERC-777, a malicious actor holding MARKET_CREATOR_ROLE could theoretically reenter the function while execution is in progress. This could cause marketCount to be incremented twice with partially initialized pool state.

The impact is rated Low because MARKET_CREATOR_ROLE is a trusted role that is controlled by governance through the Timelock. Exploitation requires a compromised role holder and a non-standard collateral token. The actual collateral deployed is USDC which does not have transfer hooks.

The proof of concept would work as follows. An attacker deploys a malicious ERC-20 token with an onTransferReceived callback. Inside that callback the attacker calls createMarket again before the first execution completes. The marketCount variable is incremented twice and the second market has an empty and inconsistent pool state.

The recommendation is to add the nonReentrant modifier to createMarket as a defense-in-depth measure, consistent with all other state-changing functions in the contract. The fix was applied by adding nonReentrant to the function signature.

L-02 — removeLiquidity Allowed While Market Is Disputed

Severity: Low **Location:** src/core/PredictionMarket.sol, lines 163 to 183 **Status:** Fixed

The removeLiquidity function did not check the market state before executing. When a market enters the Disputed state, which happens after resolveMarket is called but before the dispute window closes, liquidity providers could still withdraw their shares and take collateral out of the pool. This creates an asymmetry where traders cannot call buyOutcome because the market is no longer Open, but LPs can drain the reserves before the dispute is settled.

The impact is Low because if all collateral is removed during a dispute and the outcome is later changed, the remaining pool may not have enough USDC to pay all winning redemptions. The scenario requires coordinated timing from an LP who is aware of the dispute window.

The proof of concept is as follows. First the resolver calls resolveMarket and the state becomes PendingResolution. Then the admin calls disputeResolution and the state becomes Disputed. Then a liquidity provider calls removeLiquidity and successfully drains the collateral. The market remains stuck in Disputed state with no funds available for future winners.

The fix adds a state check at the beginning of removeLiquidity that reverts with MarketNotOpen if the market state is not Open. This is consistent with addLiquidity and buyOutcome which already had this check.

L-03 — FeeVault.setMarket Accepts Zero Address

Severity: Low **Location:** src/tokens/FeeVault.sol, setMarket function **Status:** Acknowledged

The setMarket function assigns the market state variable without validating that the provided address is non-zero. If called with address zero by accident during deployment, all subsequent depositFee calls from the real market contract would fail with the OnlyMarket error, silently breaking fee collection without a clear explanation of why.

The recommendation is to add a require statement checking that the address is not zero before assigning it. This finding is acknowledged rather than fixed because the setMarket function is only callable by the owner which is a trusted address. The deployment script sets the market address immediately after deployment and verifies it. Operational risk is low.

I-01 — Slither: block.timestamp Usage

Severity: Informational **Status:** Acknowledged

Slither's timestamp detector flags all comparisons using block.timestamp. The flagged locations in our code are the resolution time check in createMarket, the resolution time comparison in resolveMarket, the dispute window check in disputeResolution and finalizeResolution, and the staleness check in OracleAdapter.

None of these use block.timestamp as a source of randomness which would be a real vulnerability. All usages are duration comparisons where a miner or sequencer could manipulate time by approximately 12 seconds, which is completely insignificant compared to

our 3600-second staleness window and 172800-second dispute window. This is a standard and accepted pattern in DeFi protocols.

I-02 — Slither: Missing Inheritance

Severity: Informational **Status:** Acknowledged

Slither reports that PredictionMarket implements the initialize function signature declared in the IPredictionMarket interface inside MarketFactory.sol but does not explicitly inherit from it. This interface exists only to construct the abi.encodeCall selector for proxy deployment. Adding explicit inheritance would require duplicating function signatures with no security benefit.

I-03 and I-04 — Slither: constable-states and immutable-states

Severity: Informational **Status:** I-03 Acknowledged, I-04 Fixed

Slither reports that MockUSDC._decimals could be declared constant and MockAggregator._decimals could be declared immutable. Both are test and mock contracts that are never deployed to production. We updated MockAggregator._decimals to immutable as a good practice fix.

I-05 — Slither: Unindexed Event Parameters

Severity: Informational **Status:** Acknowledged

Slither flags several events in OpenZeppelin library contracts that contain address parameters without the indexed keyword. These events are in the lib directory which is outside our scope and cannot be modified.

6. Vulnerability Case Studies

Case Study 1: Reentrancy via ERC-1155 Callback

This case study demonstrates how we identified and fixed a reentrancy vulnerability in an early version of the buyOutcome function.

In the vulnerable version the code looked like this. First the outcome tokens were minted to the user with a call to outcomeToken.mint. Then the pool reserves were updated by subtracting amountOut from reserveYes. The problem is that ERC-1155 mint calls onERC1155Received on the recipient if it is a contract. A malicious contract receiving tokens could use that callback to call buyOutcome again before the reserves were updated, effectively buying tokens at the old pre-trade price.

In the fixed version we reordered the operations to follow the Checks-Effects-Interactions pattern. First all state changes happen. The pool reserves are updated, the collateral

accounting is updated, and then the external calls happen. The `safeTransferFrom` moves USDC from the user, then `depositFee` sends fees to the vault, and only then does mint issue the outcome tokens. Even if a malicious contract tries to reenter through the `onERC1155Received` callback, the `nonReentrant` modifier will cause the second call to revert immediately.

The test `test_buy_revertsSlippage` and `test_buy_feeGoesToVault` confirm that the function works correctly and the test for reentrancy confirms that reentrant calls are rejected.

Case Study 2: Missing Access Control on `resolveMarket`

This case study shows how we identified and fixed a missing authorization check on the market resolution function.

In the vulnerable version `resolveMarket` had no access control modifier. Any externally owned address could call it. The function accepts an oracle feed address stored in the market struct. If the function had no access control, an attacker could deploy a malicious `MockAggregator` that returns any price they want, then call `resolveMarket` and force any market to resolve to whatever outcome benefits them. They could buy large amounts of YES tokens, then force a YES resolution and redeem all of them.

In the fixed version we added the `onlyRole(RESOLVER_ROLE)` modifier to the function. Only addresses that have been granted the `RESOLVER_ROLE` can call `resolveMarket`. The `RESOLVER_ROLE` is assigned to the deployer at initialization and is intended to be transferred to the Timelock so that resolution requires governance approval.

The test `test_resolveMarket_revertsIfNotResolver` confirms that any call from an address without the role reverts with an `AccessControl` error.

7. Centralization Analysis

At the time of deployment, the deployer wallet holds all privileged roles. This means there is significant centralization risk before the roles are transferred to the Timelock. The table below shows what could go wrong if each role is compromised.

Role	Holder	Risk if Compromised
DEFAULT_ADMIN_ROLE	Deployer then Timelock	Can grant and revoke all other roles, can dispute resolutions
MARKET_CREATOR_ROLE	Deployer	Can create markets with malicious oracle feeds
RESOLVER_ROLE	Deployer	Can resolve markets at wrong times or with wrong feeds

PAUSER_ROLE	Deployer	Can freeze all trading indefinitely
UPGRADER_ROLE	Deployer then Timelock	Can upgrade PredictionMarket to steal all collateral
MINTER_ROLE	PredictionMarket proxy	Can mint unlimited outcome tokens if market contract is exploited

The most dangerous scenario is a compromised UPGRADER_ROLE before Timelock handover. An attacker could upgrade PredictionMarket to a malicious implementation and drain all USDC collateral from all markets in a single transaction.

After all roles are transferred to the MarketTimelock, every privileged action requires a successful governance proposal followed by a two-day waiting period. This gives the community 48 hours to react to any malicious proposal before it executes.

8. Governance Attack Analysis

Flash-Loan Governance Attack

An attacker could try to flash-borrow a large number of PRED tokens, delegate voting power to themselves, and attempt to create and pass a proposal in the same transaction. This attack is completely blocked by the ERC20Votes snapshot mechanism. Voting power for a proposal created at block N is calculated at block N minus 1. Tokens that were received and delegated in block N have zero voting power at the snapshot block. Flash-borrowed tokens cannot be used for governance.

Whale Attack

A holder who accumulates more than 4% of the total PRED supply can independently meet quorum and pass proposals without any other voter participating. At deployment, 100% of PRED is held by the treasury. Token distribution policy must ensure no single actor receives more than 4% of supply. The one-week voting period also gives other token holders time to organize and vote against malicious proposals.

Proposal Spam

A holder with the minimum threshold of 1% of supply could try to spam the governance queue with many proposals to block legitimate proposals from being processed. The OpenZeppelin Governor enforces that each proposer can only have one active proposal at a time, which significantly limits the effectiveness of spam attacks. Each spam proposal occupies the queue for approximately eight days before being defeated.

Timelock Bypass

The MarketTimelock is deployed with the EXECUTOR_ROLE granted to address zero, which in OpenZeppelin's convention means any address can call execute once the delay has elapsed. The two-day delay is enforced on the operation hash inside the TimelockController contract itself. No executor can bypass the delay by calling execute early because the contract will revert if the minimum delay has not passed.

Oracle Attack

The most realistic attack on the resolution mechanism would be to manipulate the Chainlink price feed at the exact moment of resolution to steer the binary outcome. Chainlink feeds are decentralized aggregators that source prices from many independent node operators and data providers. Manipulating the feed requires compromising a supermajority of those operators simultaneously, which is economically and technically infeasible. Our staleness check ensures we only use prices that were updated within the last hour. The answeredInRound check prevents using leftover data from previous aggregator rounds. The two-day dispute window gives the admin time to flag an incorrect resolution before it becomes final.

9. Slither Output Summary

Slither was executed with the command `slither dot` using the default detector configuration of 101 detectors. It analyzed 97 contracts in total and produced 264 results. The table below covers only findings in our project source files under `src`. All findings in `lib` are from OpenZeppelin's own intentional code patterns and are outside our scope.

Detector	Severity	Location	Action
timestamp	Low	PredictionMarket, OracleAdapter	Acknowledged — intentional usage
missing-inheritance	Informational	PredictionMarket.sol	Acknowledged — design decision
constable-states	Informational	MockUSDC._decimals	Acknowledged — mock only
immutable-states	Informational	MockAggregator._decimals	Fixed
assembly	Informational	MarketAMM.sol, MarketFactory.sol	Intentional Yul optimization
unindexed-event-address	Informational	OpenZeppelin library events	Out of scope

reentrancy-ev ents	Informationa l	OpenZeppelin Governor, OutcomeToken.mint	Library pattern, guarded by nonReentrant in callers
pragma	Informationa l	Mixed versions in lib	Library files, project uses 0.8.24 consistently

ARCHITECTURE AND DESIGN DOCUMENT

1. System Overview

This project is a decentralized prediction market protocol deployed on Arbitrum Sepolia which is an Ethereum Layer 2 network. The protocol allows any user to trade on the outcome of binary questions by buying YES or NO tokens priced automatically by an on-chain AMM. The collateral currency is USDC. When a market resolves, winners can redeem their outcome tokens for USDC at a one-to-one ratio. Trading fees flow to liquidity providers through an ERC-4626 vault. Protocol governance is handled by PRED token holders voting through an OpenZeppelin Governor with a two-day Timelock.

The protocol was built for the Blockchain Technologies 2 course final project as Option D, which requires an on-chain prediction market with AMM pricing, Chainlink oracle resolution, ERC-1155 outcome tokens, ERC-4626 fee vault, DAO governance, and Layer 2 deployment.

2. System Context

The project protocol sits at the center of several external systems that it depends on or interacts with.

End users interact with the protocol through a React frontend application. They connect their MetaMask wallet, deposit USDC, buy outcome tokens, and redeem winnings. Liquidity providers can add or remove liquidity from market pools and earn trading fees.

The Chainlink oracle network provides price feeds that the protocol uses to resolve market outcomes. For example, if a market asks whether ETH will be above three thousand dollars, the protocol reads the ETH/USD Chainlink feed at resolution time and compares the price to the threshold.

The Graph indexing network runs a subgraph that listens for events emitted by the PredictionMarket contract. The frontend reads market data, trade history, and statistics from The Graph instead of making many slow RPC calls directly to the blockchain.

Arbitrum Sepolia is the Layer 2 blockchain where all contracts are deployed. Using an L2 reduces gas costs by more than 99% compared to Ethereum mainnet, making small trades economically practical for users.

3. Container and Component Diagram

The protocol consists of ten deployed contracts. Their relationships can be described as follows.

The MarketFactory contract is the entry point for deploying new market instances. It holds the address of the PredictionMarket implementation and can deploy new ERC1967 proxy contracts using both the standard CREATE opcode and the deterministic CREATE2 opcode. Using CREATE2 allows the address of a new market to be computed before it is actually deployed.

Each deployed market proxy delegates all calls to the shared PredictionMarket implementation contract. The implementation inherits from AccessControlUpgradeable which manages four roles, PausableUpgradeable which allows emergency stops, ReentrancyGuard which prevents reentrancy attacks, and ERC1155Holder which allows the contract to receive ERC-1155 tokens. The AMM logic is provided by the MarketAMM library which is linked into the implementation.

The PredictionMarket contract interacts with three other contracts. It calls OutcomeToken to mint and burn YES and NO shares. It calls FeeVault to deposit trading fees. It calls OracleAdapter to read price feeds from Chainlink when resolving markets.

The governance system is separate from the market logic. GovernanceToken is an ERC-20 token with voting power enabled through the ERC20Votes extension and gasless approvals through ERC20Permit. GovernanceToken is also upgradeable via UUPS. PRED token holders delegate their voting power and participate in proposals through the PredictionGovernor contract. Approved proposals are queued in the MarketTimelock and executed after the two-day mandatory delay.

4. Proxy Layout

Contract	Proxy Type	Upgradeable
PredictionMarket	ERC1967 UUPS	Yes
GovernanceToken	ERC1967 UUPS	Yes

MarketFactory	None	No
FeeVault	None	No
OutcomeToken	None	No
PredictionGovernor	None	No
MarketTimelock	None	No

The two upgradeable contracts use the UUPS pattern where the upgrade authorization logic lives in the implementation rather than the proxy. This is more gas efficient than the Transparent Proxy pattern. The proxy address never changes after deployment, so users and external protocols always interact with the same address even after an upgrade.

5. Access Control Roles

Role	Contract	What It Controls
DEFAULT_ADMIN_ROLE	PredictionMarket	Grant and revoke all other roles, call disputeResolution
MARKET_CREATOR_ROLE	PredictionMarket	Call createMarket
RESOLVER_ROLE	PredictionMarket	Call resolveMarket
PAUSER_ROLE	PredictionMarket	Call pause and unpause
UPGRADER_ROLE	PredictionMarket	Authorize UUPS upgrades
MINTER_ROLE	OutcomeToken	Call mint, burn, and mintBatch
PAUSER_ROLE	OutcomeToken	Call pause and unpause
Owner	FeeVault, Factory, GovernanceToken	Call setMarket, setImplementation, and mint
PROPOSER_ROLE	MarketTimelock	Schedule operations — granted to Governor
EXECUTOR_ROLE	MarketTimelock	Execute after delay — granted to address zero meaning anyone can execute

6. Sequence Diagrams

6.1 User Flow: Buying Outcome Tokens

The `buyOutcome` function is the most frequently called function in the protocol. The following describes the complete execution path.

A user calls `buyOutcome` with four parameters: the market ID, a boolean indicating whether they want YES or NO tokens, the amount of USDC they are spending, and the minimum number of tokens they accept as output. This last parameter is slippage protection.

The function first acquires the `nonReentrant` lock to prevent reentrant calls. Then it checks that the market state is `Open` and reverts with `MarketNotOpen` if it is not. Next it calculates the fee by multiplying `amountIn` by 3 and dividing by 1000 to get exactly 0.3%. The amount after fee is `amountIn` minus the fee.

The AMM calculation runs next. The function determines `reserveIn` and `reserveOut` based on whether the user wants YES or NO tokens. Then it calls `MarketAMM.getAmountOut` using the Yul assembly path to compute how many tokens the user will receive. If the result is less than the user's `minOut` parameter the function reverts with `SlippageTooHigh`.

Next come the effects. The pool reserves are updated before any external calls. If buying YES, `reserveYes` is decreased by `amountOut` and `reserveNo` is increased by `amountInAfterFee`. The `totalCollateral` for the market is increased by `amountIn`.

Finally the interactions happen. The collateral contract transfers `amountIn` USDC from the user to the market contract. The market contract approves the fee vault for the fee amount and then calls `depositFee`. Finally `outcomeToken.mint` creates the outcome tokens and sends them to the user. An `OutcomeBought` event is emitted.

6.2 Governance Flow: Full Proposal Lifecycle

The following describes how a governance proposal moves from creation to execution.

A PRED token holder who has at least 1% of the total supply delegated to them calls `propose` on the `PredictionGovernor` with the target contract addresses, call values, encoded function call data, and a text description. The Governor reads the proposer's voting power at the previous block using the `ERC20Votes` snapshot mechanism. If the voting power meets the proposal threshold, a `ProposalCreated` event is emitted and the proposal enters the `Pending` state.

After one day which is the voting delay, the proposal becomes `Active` and token holders can cast votes. Voters call `castVote` with the proposal ID and their support value where 0 means against, 1 means for, and 2 means abstain. Voting power is calculated at the snapshot block which is the block before the proposal was created. A `VoteCast` event is emitted for each vote.

After one week which is the voting period, the proposal result is determined. If the total for votes represent at least 4% of the total PRED supply and exceed against votes, the proposal `Succeeded`. Otherwise it `Defeated`.

For a Succeeded proposal, anyone can call `queue` to send it to the Timelock. The `TimelockController` stores the operation hash and records the earliest execution timestamp as the current time plus two days. The proposal state becomes `Queued`.

After two days pass, anyone can call `execute` to run the proposal. The `TimelockController` verifies the delay has passed and then executes each call in the batch against the specified target contracts. The proposal state becomes `Executed`.

6.3 Resolution Flow: From Open Market to Redeemable Outcome

The following describes what happens when a market reaches its resolution time.

Once `block.timestamp` is greater than or equal to the market's `resolutionTime`, the `RESOLVER_ROLE` holder calls `resolveMarket`. The function verifies the caller has the role, verifies the resolution time has passed, and then calls `OracleAdapter.requireFreshPrice` with the market's oracle feed address and the maximum age of 3600 seconds.

Inside `OracleAdapter`, the `Chainlink AggregatorV3Interface` is called for the latest round data. The adapter checks that `answeredInRound` is greater than or equal to `roundId` to prevent stale data. It checks that the answer is positive. It checks that the time since `updatedAt` is within the maximum age. If any check fails the function reverts.

Back in `PredictionMarket`, the returned price is compared to the market's `resolutionPrice` threshold. If the price is greater than or equal to the threshold, the outcome is set to `YES`. Otherwise it is `NO`. The market state becomes `PendingResolution` and the `resolvedAt` timestamp is recorded. A `MarketResolved` event is emitted.

During the two-day dispute window, the `DEFAULT_ADMIN_ROLE` holder can call `disputeResolution` to move the market to `Disputed` state if they believe the resolution was incorrect. A disputed market requires a governance proposal to re-resolve.

If no dispute occurs, anyone can call `finalizeResolution` after the window expires to move the market to `Resolved` state. Once `Resolved`, winning token holders call `redeem` which burns their ERC-1155 tokens and transfers USDC at a one-to-one ratio.

7. Data Model and Storage Layout

7.1 PredictionMarket Storage

The storage layout of `PredictionMarket` must be preserved across V1 to V2 upgrades. The table below documents all slots.

Slot	Variable	Type
0	collateral	IERC20

1	outcomeToken	OutcomeToken
2	feeVault	FeeVault
3	oracle	OracleAdapter
4	marketCount	uint256
5	markets	mapping(uint256 to Market)
6	lpShares	mapping(address to mapping(uint256 to uint256))
OZ namespaced	AccessControl roles	ERC-7201 slot
OZ namespaced	Pausable state	ERC-7201 slot

OpenZeppelin upgradeable contracts use ERC-7201 namespaced storage. The slot for each inherited module is derived from a hash formula that produces a value completely independent of sequential slots 0 through 6. This means there can be no collision between our custom variables and the inherited module state. A V2 upgrade must only append new variables at slot 7 and above.

7.2 Market Struct

Each market is stored as a struct containing the following fields. The id field is a sequential integer starting from 1. The question field is a human readable string. The oracleFeed field is the address of the Chainlink aggregator. The resolutionPrice is the integer threshold for determining YES or NO outcome. The resolutionTime is the Unix timestamp after which resolution can occur. The disputeWindow is always two days. The resolvedAt field stores when resolution was called. The state field is the current MarketState enum value. The outcome field is the current Outcome enum value. The pool field contains the AMM reserves. The totalCollateral field tracks all USDC deposited into the market.

7.3 Outcome Token ID Encoding

The ERC-1155 contract uses a deterministic encoding to assign token IDs to YES and NO shares for each market. The YES token ID for a market equals the marketId multiplied by 2. The NO token ID equals marketId multiplied by 2 plus 1. This means market 1 has YES ID 2 and NO ID 3. Market 2 has YES ID 4 and NO ID 5. The encoding is collision-free for all positive market IDs and requires only one multiplication operation at runtime. A single OutcomeToken contract handles all markets without redeployment.

8. Trust Assumptions

Actor	Trust Level	Risk if Compromised
Chainlink Oracle	High, decentralized	Wrong price leads to wrong outcome. Mitigated by staleness check and dispute window
RESOLVER_ROLE holder	Trusted admin	Can resolve at wrong time or with manipulated feed. Mitigated by Timelock handover
UPGRADER_ROLE holder	Trusted Timelock	Can upgrade to malicious implementation and steal all collateral
Timelock and Governor majority	Governance, needs 50% plus quorum	Malicious proposal passes. Two-day delay provides response window
USDC Circle	Highly trusted	Circle could blacklist the market contract address, freezing all collateral
L2 Sequencer	Trusted	Sequencer could censor resolution transactions. Mitigated by decentralized sequencer roadmap
The Graph	Semi-trusted, off-chain	Indexer could show stale data. Frontend falls back to direct RPC reads for critical state

The Timelock controls the most powerful actions in the protocol including upgrading contracts, transferring treasury ownership, and changing governance parameters. Before handover the deployer wallet has these powers directly. After handover every action requires eight days minimum: one day voting delay plus seven days voting period plus two days Timelock delay.

9. Design Patterns

The following ten patterns are implemented in the protocol. Each one was chosen because it solves a real problem in the design.

Pattern	Location	Reason
UUPS Proxy	PredictionMarket, GovernanceToken	Post-deployment bug fixes without fund migration
Factory CREATE and CREATE2	MarketFactory	Deterministic addresses and auditable deployment history

Checks-Effects-Interactions	All state-changing functions	Prevents reentrancy from ERC-1155 callbacks
Reentrancy Guard	buyOutcome, addLiquidity, removeLiquidity, redeem	Defense in depth on top of CEI pattern
Access Control	PredictionMarket, OutcomeToken	Separate keys for separate responsibilities
Pausable	PredictionMarket, OutcomeToken	Emergency stop for new activity without freezing existing state
State Machine	Market.state with five states	Enforces correct lifecycle, invalid actions revert deterministically
Oracle Adapter	OracleAdapter.sol	Decouples Chainlink from market logic, centralized staleness checks
Timelock	MarketTimelock	Mandatory delay between governance approval and execution
Pull-over-Push	redeem function	Winners initiate their own payouts, no unsolicited fund transfers

10. Architecture Decision Records

ADR-01: CPMM Instead of LMSR for Outcome Pricing

The team considered two pricing mechanisms. The Logarithmic Market Scoring Rule provides bounded loss for the market maker and ensures prices sum to one by construction. However it requires computing logarithms on-chain which is expensive in gas and complex to audit. The constant-product market maker uses the identical mathematics as Uniswap V2, which is extensively audited in practice, easy to understand, and natively supports LP share issuance.

We chose CPMM. The simplicity and auditability advantages outweigh the theoretical loss bound benefit of LMSR. The drain risk from large trades is mitigated by requiring non-zero initial liquidity and by the minOut slippage parameter on every trade. The invariant test `invariant_k_neverDecreases` running for 500 rounds of 50000 handler calls provides continuous assurance.

ADR-02: ERC-1155 Instead of ERC-20 Pairs for Outcome Tokens

For two token types per market there are two options. We could deploy two separate ERC-20 contracts for YES and NO tokens per market, which would be composable with existing DeFi but would require two deployments per market. Alternatively we could use one ERC-1155 contract for all markets using ID encoding.

We chose ERC-1155. With the ID encoding scheme of marketId times 2 for YES and plus 1 for NO, one contract deployment serves unlimited markets. The tradeoff is that outcome tokens cannot be directly used in external protocols like Uniswap without a wrapper, but this is acceptable because prediction market tokens are primarily meant for redemption not secondary trading.

ADR-03: Yul Assembly in getAmountOut

The swap calculation runs on every trade which makes it the hot path in the protocol. We implemented it in Yul inline assembly to reduce gas by approximately 12% per call. We kept a pure Solidity reference implementation called getAmountOutPure alongside it. The fuzz test testFuzz_yul_equalsPure runs 1000 times across the full uint64 input range and confirms that both implementations return identical results in all cases.

ADR-04: V1 to V2 Upgrade Path

For any future V2 upgrade the rules are as follows. Storage slots 0 through 6 as documented in Section 7.1 are frozen permanently. No existing variable can be deleted, reordered, or changed to a different type. New variables must be added starting at slot 7. The upgrade transaction must be proposed through the full governance process, wait the two-day Timelock delay, and then execute. Before the proxy is pointed to the new implementation, a verification script must confirm the storage layout of V2 against the documented baseline.

11. Gas Optimization Report

Yul vs Pure Solidity Benchmark

The getAmountOut function is called on every buyOutcome transaction. Implementing it in Yul assembly instead of pure Solidity saves approximately 513 gas per call, which is a 12% reduction. The table below shows the average gas from the fuzz test suite.

Implementation	Average Gas Per Call
getAmountOut in Yul assembly	4609 gas
getAmountOutPure in pure Solidity	5122 gas
Gas saving	513 gas per call (12%)

Key Function Gas Costs

Function	Average Gas
-----------------	--------------------

buyOutcome	113758 gas
createMarket	398841 gas
addLiquidity	98831 gas
removeLiquidity	81855 gas
resolveMarket	56237 gas
redeem	36154 gas
castVote	76617 gas
propose	78044 gas
queue	112163 gas
execute	83979 gas

L1 vs L2 Gas Cost Comparison

All contracts are deployed on Arbitrum Sepolia which is an Ethereum Layer 2 rollup. The execution gas costs are the same as Ethereum but the cost per unit of gas is dramatically lower because the L2 batches many transactions together and compresses the calldata before posting it to L1.

Operation	Estimated L1 Cost at 30 gwei	Estimated L2 Cost on Arbitrum	Saving
buyOutcome	approximately \$10	less than \$0.01	over 99%
createMarket	approximately \$38	less than \$0.01	over 99%
castVote	approximately \$7	less than \$0.01	over 99%
propose	approximately \$7	less than \$0.01	over 99%
redeem	approximately \$4	less than \$0.01	over 99%
addLiquidity	approximately \$9	less than \$0.01	over 99%

The L1 estimates assume 30 gwei gas price and ETH at \$3000. The L2 estimates are based on observed gas prices of approximately 0.01 gwei on Arbitrum Sepolia testnet during

deployment. Deploying on L2 makes the protocol economically practical for real users. On Ethereum mainnet a single buyOutcome call would cost around \$10 which would be unacceptable for a prediction market where users might trade \$10 total.